

Tugas 6: Deadlock

Dosen Pengampu :

TRIAWAN ADI CAHYANTO M.Kom



Disusun Oleh:

Pradana Bagus Susilo (2410651007)

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

Tujuan

1. Memahami Konsep Dasar Sistem Operasi

- Menjelaskan fungsi utama sistem operasi, seperti manajemen proses, memori, dan perangkat keras.
- Membedakan antara kernel, shell, dan distribusi Linux (misalnya, Ubuntu, CentOS).
- Mengidentifikasi komponen utama Linux, termasuk file system (ext4, Btrfs) dan boot process.

2. Menguasai Command Line Interface (CLI)

- Menggunakan perintah dasar seperti ls, cd, mkdir, cp, mv, dan rm untuk navigasi dan manipulasi file.
- Memahami penggunaan pipes, redirection, dan wildcards untuk efisiensi kerja.
- Menjalankan perintah dengan opsi lanjutan, seperti grep, find, dan sed untuk pencarian dan pengeditan teks.

3. Mengelola File dan Direktori

- Mengatur izin file dan direktori menggunakan chmod, chown, dan chgrp.
- Memahami struktur file system Linux (/root, /home, /etc, dll.) dan mounting perangkat.
- Menggunakan alat seperti tar, gzip, dan rsync untuk kompresi dan sinkronisasi file.

4. Menginstal dan Mengkonfigurasi Software

- Menggunakan package manager seperti apt (Debian-based) atau yum/dnf (Red Hat-based) untuk instalasi aplikasi.
- Mengkompilasi software dari source code menggunakan make dan configure.
- Mengelola dependensi dan repositori perangkat lunak.

5. Mengelola Proses dan Pengguna

- Memantau dan mengontrol proses dengan ps, top, kill, dan nice.
- Membuat dan mengelola akun pengguna serta grup menggunakan useradd, usermod, dan passwd.
- Mengonfigurasi layanan sistem seperti systemd untuk startup otomatis.

6. Dasar-Dasar Jaringan dan Keamanan

- Mengkonfigurasi jaringan dengan ifconfig atau ip, termasuk pengaturan IP, DNS, dan firewall (iptables atau firewalld).
- Memahami protokol dasar seperti SSH untuk akses jarak jauh dan SCP untuk transfer file.

Tugas 1: Analisis Deadlock

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Jawaban:

Soal 1:

Program mengalami deadlock karena kedua thread saling menunggu mutex yang di-lock oleh thread lain, dan tidak ada yang bisa melanjutkan eksekusi.

Urutannya:

Thread 1

1. Mendapat Lock 1
2. Sleep 1 detik
3. Mencoba mengambil Lock 2, tetapi
 - Lock 2 sedang dipegang Thread 2 → Thread 1 menunggu

Thread 2

1. Mendapat Lock 2
2. Sleep 1 detik
3. Mencoba mengambil Lock 1, tetapi
 - Lock 1 sedang dipegang Thread 1 → Thread 2 menunggu

Kondisi akhir

- Thread 1 memegang Lock1 → menunggu Lock2
- Thread 2 memegang Lock2 → menunggu Lock1

Mereka saling menunggu selamanya → deadlock.

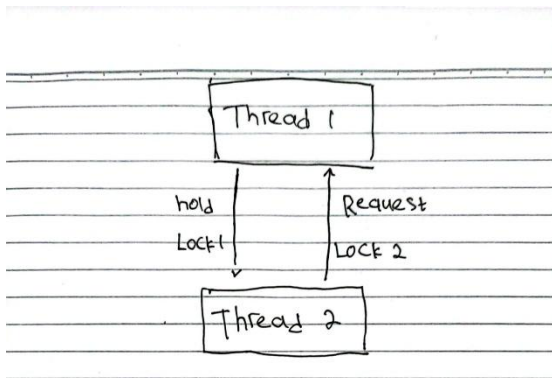
Soal 2:

Kondisi deadlock yang terpenuhi:

1. Mutual Exclusion
Lock adalah resource eksklusif—hanya 1 thread yang dapat memegangnya.
2. Hold and Wait
Thread 1 memegang Lock1 dan menunggu Lock2, Thread 2 memegang Lock2 dan menunggu Lock1.
3. No Preemption
Mutex tidak dapat dipaksa direbut; thread harus me-release sendiri.
4. Circular Wait
Thread 1 → menunggu Lock2 → dimiliki Thread 2 → menunggu Lock1 → dimiliki Thread 1.

Soal 3:

Diagram Resource Allocation Graph



Tugas 2: Implementasi Pencegahan

Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

Jawaban

Soal 1: Lock ordering

Source Kode:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_func(void* arg) {  
    printf("Thread 1: Mengambil Lock1...\n");  
    pthread_mutex_lock(&lock1);
```

```
    sleep(1);
```

```
    printf("Thread 1: Mengambil Lock2...\n");  
    pthread_mutex_lock(&lock2);
```

```
    printf("Thread 1: Selesai bekerja\n");
```

```
    pthread_mutex_unlock(&lock2);
```

```

pthread_mutex_unlock(&lock1);

return NULL;
}

void* thread2_func(void* arg) {
    printf("Thread 2: Mengambil Lock1...\n");
    pthread_mutex_lock(&lock1); // DULUNYA dia ambil Lock2 dulu — diganti!

    sleep(1);

    printf("Thread 2: Mengambil Lock2...\n");
    pthread_mutex_lock(&lock2);

    printf("Thread 2: Selesai bekerja\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);

    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== Lock Ordering ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== Program Selesai ===\n");
    return 0;
}

```

Output:

```

pradana@Bagus:~$ ./deadlock_simple
=== Lock Ordering ===

Thread 1: Mengambil Lock1...
Thread 2: Mengambil Lock1...
Thread 1: Mengambil Lock2...
Thread 1: Selesai bekerja
Thread 2: Mengambil Lock2...
Thread 2: Selesai bekerja

=== Program Selesai ===
pradana@Bagus:~$ |

```

Soal 2: Try-lock dengan retry

Source Kode:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread1_func(void* arg) {
    while (1) {
        printf("Thread 1: Mencoba Lock1...\n");
        pthread_mutex_lock(&lock1);

        sleep(1);

        printf("Thread 1: Mencoba Lock2...\n");
        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: Berhasil lock dua-duanya!\n");
            break; // sukses
        }

        printf("Thread 1: Gagal Lock2 → melepas Lock1 dan retry...\n");
        pthread_mutex_unlock(&lock1);
        usleep(100000); // 100 ms backoff
    }

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

void* thread2_func(void* arg) {
    while (1) {
        printf("Thread 2: Mencoba Lock2...\n");
        pthread_mutex_lock(&lock2);

        sleep(1);

        printf("Thread 2: Mencoba Lock1...\n");
        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: Berhasil lock dua-duanya!\n");
            break;
        }
    }
}
```

```

        printf("Thread 2: Gagal Lock1 → melepas Lock2 dan retry...\n");
        pthread_mutex_unlock(&lock2);
        usleep(100000);
    }

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== Try-Lock with Retry ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\n=== Program Selesai ===\n");
    return 0;
}

```

Source Code:

```

pradana@Bagus:~$ gcc deadlock_simple.c -o deadlock_simple -lpthread
pradana@Bagus:~$ ./deadlock_simple
=== Try-Lock with Retry ===

Thread 1: Mencoba Lock1...
Thread 2: Mencoba Lock2...
Thread 2: Mencoba Lock1...
Thread 2: Gagal Lock1 → melepas Lock2 dan retry...
Thread 1: Mencoba Lock2...
Thread 1: Berhasil lock dua-duanya!
Thread 2: Mencoba Lock2...
Thread 2: Mencoba Lock1...
Thread 2: Berhasil lock dua-duanya!

=== Program Selesai ===
pradana@Bagus:~$ |

```

Soal 3: Bandingkan kedua metode tersebut

1. Lock Ordering → cara terbaik, deterministik, zero deadlock.
2. Try-lock Retry → bagus jika ingin sistem tetap non-blocking, tetapi bisa boros CPU dan tidak deterministik.

Tugas 3: Banker's Algorithm

Buat program baru dengan data:

4 proses

3 jenis resource

Available: [5, 4, 3]

Implementasikan request resource dan cek keamanan system

Jawaban:

Source Kode:

```
#include <stdio.h>

#define P 4 // jumlah proses
#define R 3 // jumlah jenis resource

int available[R] = {5, 4, 3}; // Resources tersedia
int max[P][R] = {
    {7, 5, 3},
    {3, 2, 2},
    {9, 0, 2},
    {2, 2, 2}
};

int allocation[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1}
};

int need[P][R];

// -----
// Hitung matriks Need
// -----
void calculate_need() {
    for (int i = 0; i < P; i++)
        for (int j = 0; j < R; j++)
            need[i][j] = max[i][j] - allocation[i][j];
}

// -----
// CEK STATE AMAN
// -----
int is_safe() {
```

```

int work[R], finish[P] = {0};

for (int i = 0; i < R; i++)
    work[i] = available[i];

int count = 0;
int safe_sequence[P];

while (count < P) {
    int found = 0;

    for (int p = 0; p < P; p++) {
        if (!finish[p]) {
            int can_run = 1;
            for (int j = 0; j < R; j++) {
                if (need[p][j] > work[j]) {
                    can_run = 0;
                    break;
                }
            }

            if (can_run) {
                for (int j = 0; j < R; j++)
                    work[j] += allocation[p][j];

                safe_sequence[count++] = p;
                finish[p] = 1;
                found = 1;
            }
        }
    }

    if (!found) { // tidak ada proses yang bisa jalan → tidak aman
        printf("\n>>> STATE TIDAK AMAN! <<<\n");
        return 0;
    }
}

printf("\nSAFE SEQUENCE: ");
for (int i = 0; i < P; i++)
    printf("P%d ", safe_sequence[i]);

printf("\n>>> STATE AMAN <<<\n");
return 1;
}

```

```

// -----
// REQUEST RESOURCE
// -----
void request_resources(int p, int req[]) {
    printf("\nProses P%d meminta resources: [%d %d %d]\n",
        p, req[0], req[1], req[2]);

    // Cek apakah request ≤ need
    for (int i = 0; i < R; i++)
        if (req[i] > need[p][i]) {
            printf("ERROR: request melebihi need!\n");
            return;
        }

    // Cek apakah resource tersedia
    for (int i = 0; i < R; i++)
        if (req[i] > available[i]) {
            printf("ERROR: resource tidak cukup, P%d harus menunggu.\n", p);
            return;
        }

    // Simulasikan alokasi SEMENTARA (banker's algorithm)
    for (int i = 0; i < R; i++) {
        available[i] -= req[i];
        allocation[p][i] += req[i];
        need[p][i] -= req[i];
    }

    // Cek apakah sistem aman setelah alokasi
    if (!is_safe()) {
        printf("Request P%d DITOLAK karena menyebabkan state tidak aman.\n", p);

        // rollback
        for (int i = 0; i < R; i++) {
            available[i] += req[i];
            allocation[p][i] -= req[i];
            need[p][i] += req[i];
        }
    } else {
        printf("Request P%d DISETUJUI.\n", p);
    }
}

// -----

```

```

int main() {
    calculate_need();

    printf("=== SIMULASI BANKER ALGORITHM ===\n");

    printf("\nAvailable awal: [%d %d %d]\n", available[0], available[1], available[2]);

    printf("\nMengecek initial safety...\n");
    is_safe();

    // Contoh request: proses 1 meminta resource
    int request1[R] = {1, 0, 2};
    request_resources(1, request1);

    // Contoh request lain
    int request2[R] = {3, 3, 0};
    request_resources(0, request2);

    return 0;
}

```

Output:

```

pradana@Bagus:~$ nano banker.c
pradana@Bagus:~$ gcc banker.c -o banker
./banker
=== SIMULASI BANKER ALGORITHM ===

Available awal: [5 4 3]

Mengecek initial safety...

SAFE SEQUENCE: P1 P2 P3 P0
>>> STATE AMAN <<<

Proses P1 meminta resources: [1 0 2]

SAFE SEQUENCE: P1 P2 P3 P0
>>> STATE AMAN <<<
Request P1 DISETUJUI.

Proses P0 meminta resources: [3 3 0]

SAFE SEQUENCE: P3 P1 P2 P0
>>> STATE AMAN <<<
Request P0 DISETUJUI.
pradana@Bagus:~$ |

```

Tugas 4: Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

Orang A: transfer dari Rekening 1 ke Rekening 2

Orang B: transfer dari Rekening 2 ke Rekening 1

Implementasikan deadlock prevention

Jawaban:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Dua rekening, masing-masing punya mutex
int rekening1 = 1000;
int rekening2 = 1000;

pthread_mutex_t lockR1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lockR2 = PTHREAD_MUTEX_INITIALIZER;

// -----
// SIMULASI DEADLOCK: A dan B mengambil lock berlawanan
// -----
void* transferA_deadlock(void* arg) {
    printf("[A] Mengunci Rekening 1...\n");
    pthread_mutex_lock(&lockR1);
    sleep(1);

    printf("[A] Mengunci Rekening 2...\n");
    pthread_mutex_lock(&lockR2);

    rekening1 -= 100;
    rekening2 += 100;

    pthread_mutex_unlock(&lockR2);
    pthread_mutex_unlock(&lockR1);
    return NULL;
}

void* transferB_deadlock(void* arg) {
    printf("[B] Mengunci Rekening 2...\n");
    pthread_mutex_lock(&lockR2);
    sleep(1);

    printf("[B] Mengunci Rekening 1...\n");
```

```

pthread_mutex_lock(&lockR1);

rekening2 -= 150;
rekening1 += 150;

pthread_mutex_unlock(&lockR1);
pthread_mutex_unlock(&lockR2);
return NULL;
}

// -----
// DEADLOCK PREVENTION (Lock Ordering)
// Semua proses lock rekening berdasarkan ID terkecil dulu
// -----
void lock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    if (a < b) {           // pointer comparing → ordering
        pthread_mutex_lock(a);
        pthread_mutex_lock(b);
    } else {
        pthread_mutex_lock(b);
        pthread_mutex_lock(a);
    }
}

void unlock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    if (a < b) {
        pthread_mutex_unlock(b);
        pthread_mutex_unlock(a);
    } else {
        pthread_mutex_unlock(a);
        pthread_mutex_unlock(b);
    }
}

void* transferA_safe(void* arg) {
    printf("[A] Transfer aman (1 → 2)\n");
    lock_ordered(&lockR1, &lockR2);

    rekening1 -= 200;
    rekening2 += 200;

    unlock_ordered(&lockR1, &lockR2);
    return NULL;
}

```

```

void* transferB_safe(void* arg) {
    printf("[B] Transfer aman (2 → 1)\n");
    lock_ordered(&lockR1, &lockR2);

    rekening2 -= 300;
    rekening1 += 300;

    unlock_ordered(&lockR1, &lockR2);
    return NULL;
}

// -----
// MAIN PROGRAM
// -----
int main() {

    pthread_t tA, tB;

    printf("\n=== SIMULASI DEADLOCK ===\n");
    pthread_create(&tA, NULL, transferA_deadlock, NULL);
    pthread_create(&tB, NULL, transferB_deadlock, NULL);

    // Ini akan macet (deadlock)
    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    printf("\nSaldo akhir (deadlock, tidak pernah tampil): R1=%d R2=%d\n",
        rekening1, rekening2);

    printf("\n=== SIMULASI DEADLOCK PREVENTION ===\n");

    // Reset saldo
    rekening1 = 1000;
    rekening2 = 1000;

    pthread_create(&tA, NULL, transferA_safe, NULL);
    pthread_create(&tB, NULL, transferB_safe, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    printf("\nSaldo akhir setelah pencegahan: R1=%d R2=%d\n",
        rekening1, rekening2);
}

```

```
printf("\n=== PROGRAM SELESAI ===\n");  
return 0;  
}
```

Output:

```
pradana@Bagus:~$ nano transfer_deadlock.c  
pradana@Bagus:~$ gcc transfer_deadlock.c -o transfer_deadlock -l  
pthread  
pradana@Bagus:~$ ./transfer_deadlock  
  
=== SIMULASI DEADLOCK ===  
[A] Mengunci Rekening 1...  
[B] Mengunci Rekening 2...  
[B] Mengunci Rekening 1...  
[A] Mengunci Rekening 2...
```